

Formalizing Combinatorics: A Comparative Review of Lean 4 and Isabelle/HOL

Miaocong He

Introduction

This document serves as the documentation component of my final project, which formalized the Handshaking Lemma, a fundamental combinatorial theorem stating that the sum of vertex degrees in a finite graph equals twice the number of edges. Having implemented the proof architecture in Lean 4, I will use my codebase as a practical benchmark to evaluate the philosophical and technical trade-offs between Lean 4 and Isabelle/HOL. Drawing upon both mathematical logic and computer science principles, this review focuses on library design, automation ergonomics, and the handling of discrete finite structures.

Type Theory vs. Higher-Order Logic: The Quotient Structure Bottleneck

The most pronounced difference between the two systems lies in their foundational logic. Lean 4 is built upon Dependent Type Theory (DTT), which provides immense expressive power but strictly enforces type hierarchies. During my formalization, this strictness manifested as a significant technical bottleneck. In Lean’s `Mathlib`, undirected edges are represented using `Sym2 V` (unordered pairs constructed via quotient types). Unpacking this quotient type to compute cardinalities within a finite sum (`Finset.sum`) introduced substantial type-coercion overhead. To preserve the combinatorial flow of the project, I opted to isolate this behavior axiomatically.

Conversely, Isabelle is based on Higher-Order Logic (HOL). In Isabelle’s Archive of Formal Proofs (AFP), graphs are frequently defined using simpler set-theoretic approximations, such as symmetric relations or sets of two-element sets. Because HOL does not enforce the same strict, dependent type hierarchies as Lean, managing the algebraic translation of unordered pairs in Isabelle is often less syntactically demanding. Isabelle trades the profound architectural safety of DTT for a more relaxed, “paper-math” approach to simple sets.

Automation Ergonomics: Sledgehammer vs. Tactics

A secondary point of comparison is the developer experience regarding proof automation. My Lean implementation originally attempted to use the `omega` tactic (a Presburger arithmetic decision procedure) to resolve algebraic conversions (e.g., $n \times 2 = 2 \times n$). However, the underlying ring-theory lemmas introduced type-casts (coercions) from natural numbers to abstract ring elements, causing `omega` to fail. I ultimately resolved this using Lean’s term-rewriting simplifier (`simp [mul_comm]`), which executes proofs dynamically at the syntax-tree level without structural pollution.

Had this project been executed in Isabelle/HOL, the experience would have relied heavily on **Sledgehammer**. Sledgehammer operates completely differently: it translates the goal and passes it to external automated theorem provers (like Z3, CVC4, and E). In Isabelle, the step verifying that the edge incidences sum to $2|E|$ would likely be solved entirely automatically with a single `by auto` or `by sledgehammer` command. While Isabelle excels at hiding the low-level algebraic mechanics, Lean 4’s tactic-based approach forced a deeper engagement with the actual algorithms governing type resolution, a highly valuable exercise from a computer science perspective.

Library Ecosystem: Monolith vs. Archive

Finally, the architectural design of the libraries dictates the formalization strategy. Lean’s **Mathlib** is monolithic; it is a single, highly integrated hierarchy. By importing **Mathlib**, my Handshaking Lemma instantly gained access to the global **Algebra.BigOperators** and generic **Finset** behaviors. However, this monolithic nature means changes to base definitions (like **Fintype**) affect the entire ecosystem. Isabelle’s AFP, by contrast, is a decentralized collection of modules. While this allows for multiple, competing definitions of a “Graph” in Isabelle, it lacks the uniform, global algebraic integration that makes Lean’s Mathlib the gold standard for modern pure mathematics.

Conclusion

Both platforms offer exceptional capabilities for formal verification. Isabelle/HOL, fortified by Sledgehammer, remains the pragmatic choice for rapid verification of isolated algorithms or when set-theoretic simplicity is preferred. However, Lean 4, with its Dependent Type Theory and integrated Mathlib, excels in constructing deep, structurally rigorous mathematical theories. Completing the Handshaking Lemma in Lean 4 demonstrated that while DTT imposes a steep initial learning curve, particularly regarding quotients and finite sets, it ultimately provides unparalleled control over the computational and logical integrity of the proof.